

File Operations in C

SCS 1301 - Data Structures and Program Design in C

Dilanga Harshani

University of Colombo School of Computing

July 9, 2026

Overview

- Why do we need file handling?
- The FILE pointer
- Opening and closing files
- Reading from files
- Writing to files
- Appending to files
- r+, w+, a+ modes and how they differ
- fseek(), ftell() and rewind() functions

Why Do We Need File Handling?

- Everything your program has done so far disappears when it ends
- Files let data **persist** beyond a single run of the program
- Programs need to:
 - Save results (logs, records, configuration)
 - Read input that's too large to type in every time
 - Share data between two different programs

Quick Recap: Pointers

- A pointer is a variable that stores an **address**, not a value directly
- Pointer types matter, an `int *` cannot correctly point to a `float`

```
int x = 10;  
int *p = &x;    // p stores the address of x  
printf("%d", *p); // dereference: gives us 10
```

- A `FILE *` is just a pointer too, instead of pointing to an `int` or `float`, it points to a special struct that keeps track of an open file. That's why its type must always be `FILE *`.

The FILE Pointer

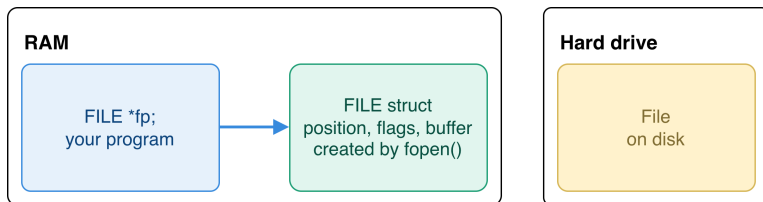
- Declared like any other pointer, but with type `FILE`

```
FILE *fp;
```

- `FILE` is not a basic type like `int`, it is a **struct typedef** defined inside the C standard library `stdio.h`
- It stores various information such as the current position in the file, whether an error or end-of-file occurred and buffering information.

The FILE Pointer

- Your program's FILE `*fp` points to a FILE struct maintained by the C library



Opening a File

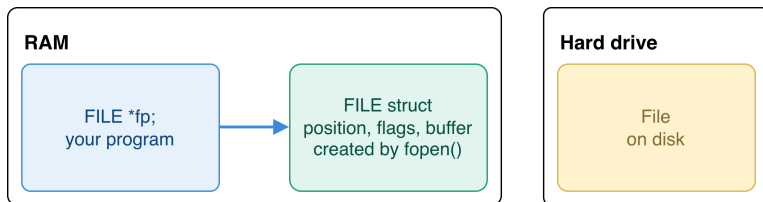
```
FILE *fopen(const char *filename, const char *mode);
```

- Returns a FILE * pointing to the opened file
- Returns NULL if the file could not be opened

```
FILE *fp = fopen("data.txt", "r");  
if (fp == NULL) {  
    printf("Could not open file\n");  
    return 1;  
}
```

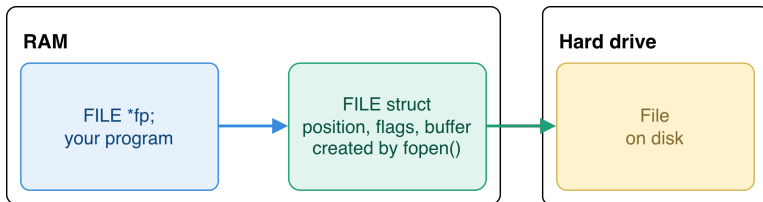
Opening a File

- Your program's FILE `*fp` points to a FILE struct maintained by the C library



Opening a File

- That struct internally tracks the actual file on disk, plus a position marker



The Six File Modes

- `fopen` takes a mode as its second argument, which decides up to what extent the file can be used

Single-purpose

- `r` : read only
- `w` : write only
- `a` : append (write only)

Read + write

- `r+` : read and write
- `w+` : write and read
- `a+` : read and append

Closing a File

```
fclose(fp);
```

- Writes may be **buffered**, not immediately written to disk
- `fclose()` flushes this buffer, guaranteeing your data is saved
- Releases the OS resource
- Not closing opened files can cause,
 - lost/incomplete data
 - resource leaks in long-running programs

“r” : Read Only Mode

```
fopen(..., "r")
```

- File must already exist before reading in “r” mode
- Functions
 - `fgetc()` reads one character at a time
 - `fgets()` reads one line at a time
 - `fscanf()` reads formatted/structured data

Reading One Character with fgetc()

```
int fgetc(FILE *fp);
```

- Reads a single character and returns it
- Returns the constant EOF when there is nothing left to read

```
FILE *fp = fopen("data.txt", "r");  
char ch;  
ch = fgetc(fp);  
printf("%c", ch);  
fclose(fp);
```

Reading Until EOF with fgetc()

```
FILE *fp = fopen("data.txt", "r");
char ch;
while ((ch = fgetc(fp)) != EOF) {
    putchar(ch);
}
fclose(fp);
```

Reading a Line with fgets()

```
char *fgets(char *str, int n, FILE *fp);
```

- Reads up to n-1 characters, or until a newline character
- Stores a '\0' at the end automatically (Refer string operations)

```
char str[30];  
FILE *fp = fopen("data.txt", "r");  
fgets(str, 10, fp); // Reads 9 characters  
printf("%s", str);  
fclose(fp);
```

- The newline character is included in the length if it was read
- If the line is longer than the given buffer length, fgets stops

Formatted Reading with fscanf()

```
int fscanf(FILE *fp, const char *format, ...);
```

- Same rules as `scanf`, just add the stream as the first argument
- Useful when the file has a predictable, structured format

```
FILE *fp = fopen("data.txt", "r");
int id;
char name[50];
while (fscanf(fp, "%d %s", &id, name) != EOF) {
    printf("ID: %d, Name: %s\n", id, name);
}
fclose(fp);
```


“w” : Write Mode

`fopen(..., "w")`

- If the file exists, its contents are overwritten
- If the file does not exist, it is created
- Functions
 - `fputc()` writes one character at a time
 - `fputs()` writes a string
 - `fprintf()` writes formatted data

Writing One Character with fputc()

```
int fputc(int ch, FILE *fp);
```

- Writes a single character into the file

```
FILE *fp = fopen("output.txt", "w");  
fputc('H', fp);  
fclose(fp);
```

Writing a String with fputc()

```
char str[] = "Hello, File!";  
FILE *fp = fopen("output.txt", "w");  
  
int i = 0;  
while (str[i] != '\0') {  
    fputc(str[i], fp);  
    i++;  
}  
fclose(fp);
```

Writing a String with fputs()

```
int fputs(const char *str, FILE *fp);
```

- fputs() does not automatically add a newline, should be added with "\n"

```
FILE *fp = fopen("output.txt", "w");  
fputs("Hello, File!\n", fp);  
fclose(fp);
```

Formatted Writing with fprintf()

```
int fprintf(FILE *fp, const char *format, ...);
```

- Same as printf, but writes to a file instead of the screen

```
FILE *fp = fopen("output.txt", "w");  
fprintf(fp, "Name: %s\nAge: %d\n", "Anne", 20);  
fclose(fp);
```

“a” : Append Mode

```
fopen(..., "a")
```

- If the file exists, new data is added after existing content
- If the file does not exist, it is created
- Same functions as "w" mode, but every write lands at the end
- Functions
 - `fputc()`, appends one character at the end
 - `fputs()`, appends a string at the end
 - `fprintf()`, appends formatted data at the end

Append to a Log File with “a” mode

```
FILE *fp = fopen("log.txt", "a");  
fprintf(fp, "New entry\n");  
fclose(fp);
```

- The append will be added to the same line if the newline character "\n" is not specified

Mode Comparison

Mode	If file exists	If file missing	Access
r	opens, keeps content	error (NULL)	read only
w	overwrites file content	creates file	write only
a	writes at the end	creates file	write only (append)

“r+” : Reading and Writing Mode

```
fopen(..., "r+")
```

- Opens an existing file for both reading and writing
- Does not create the file if it does not exist.
- Does not overwrite existing content
- Functions
 - `fgetc()`, `fgets()`, `fscanf()` for reading
 - `fputc()`, `fputs()`, `fprintf()` for writing
 - `fseek` for moving to a specific position before reading or writing

Writing to a Specific Position in a File

```
FILE *fp = fopen("data.txt", "r+");  
if (fp == NULL) {  
    printf("File must already exist for r+\n");  
    return 1;  
}  
  
fseek(fp, 5, SEEK_SET); // move to the 6th byte  
fputs("XXXXX", fp);     // overwrite 5 characters  
fclose(fp);
```

- Only the bytes at that position are overwritten, everything else in the file stays untouched

Moving the File Position with fseek()

```
int fseek(FILE *fp, long offset, int position);
```

- Moves the file's internal position marker, so the next read/write happens from a different spot
- Position tells fseek() to start counting from
 - SEEK_SET from the beginning of the file
 - SEEK_CUR from the current position
 - SEEK_END from the end of the file

```
fseek(fp, 0, SEEK_SET);    // go to the very start  
fseek(fp, 5, SEEK_CUR);    // move 5 bytes from here  
fseek(fp, 0, SEEK_END);    // go to the very end
```

Finding the Current Position with ftell()

```
long ftell(FILE *fp);
```

- Returns the current position (in bytes) from the start of the file
- Useful to check position, or to measure file size

```
FILE *fp = fopen("data.txt", "r");  
fseek(fp, 0, SEEK_END);  
long size = ftell(fp);    // gives file size in bytes  
printf("File size: %ld bytes\n", size);  
fclose(fp);
```

“w+” : Writing and Reading Mode

```
fopen(..., "w+")
```

- Opens a file for both reading and writing
- Overwrites an existing file or creates if it does not exist
- Difference from "w": Can also read back what is written
- Difference from "r+": "w+" always truncates or creates
- Functions
 - `fputc()`, `fputs()`, `fprintf()` for writing
 - `fgetc()`, `fgets()`, `fscanf()` for reading
 - `fseek` for moving to a specific position before reading or writing
 - `rewind()` for moving back to the start

Move Back to the Start with `rewind()`

```
FILE *fp = fopen("data.txt", "w+");
fputs("Hello World", fp);

rewind(fp);    // move position back to the start

char buffer[50];
fgets(buffer, 50, fp);
printf("%s\n", buffer);
fclose(fp);
```

- After writing, the file position is at the end *Trying to read immediately would just hit EOF*
- *`rewind(fp)` resets the position to the beginning, equivalent to `fseek(fp, 0, SEEK_SET)`*

“a+” : Reading and Appending Mode

- `fopen(..., "a+")`
- Opens a file for both reading and appending
- If the file exists, content is preserved and reading is allowed anywhere, but all writes still go to the end
- If the file does not exist, it is created
- Functions are same as "r+" and "w+" modes, but writes are always added to the end of the file

Reading and Appending a Log File with “a+” Mode

```
FILE *fp = fopen("log.txt", "a+");  
fprintf(fp, "New log entry\n");  
  
rewind(fp);  
char buffer[100];  
while (fgets(buffer, 100, fp) != NULL) {  
    printf("%s", buffer);  
}  
fclose(fp);
```


Mode Comparison

Mode	If file exists	If file missing	Access
r	opens, keeps content	error (NULL)	read only
w	overwrites file content	creates file	write only
a	keeps, writes at the end	creates file	write only (append)
r+	keeps content	error (NULL)	read + write
w+	overwrites file content	creates file	read + write
a+	keeps, writes at the end	creates file	read + write (append)

Summary

- `FILE*` : a pointer to a struct that tracks an open file
- `fopen/fclose` : always open, always check for NULL, always close
- Reading from a file : `fgetc()`, `fgets()`, `fscanf()`
- Writing to a file : `fputc()`, `fputs()`, `fprintf()`
- Six modes `r`, `w`, `a`, `r+`, `w+`, `a+` : differ in whether they create / overwrite, and whether they allow read, write, or both
- `fseek()` / `ftell()` / `rewind()` : move around and check position within a file